

CET138 JavaScript: The Language of the Web

Introduction

JavaScript is a programming language that runs in web browsers and enables dynamic, interactive web experiences. While HTML provides structure and CSS provides visual design, JavaScript brings websites to life through interactivity, DOM manipulation, event handling, and complex logic.

Key JavaScript concepts include:

- DOM Manipulation: Accessing and modifying HTML elements dynamically.
- Event Handling: Responding to user interactions like clicks, form submissions, and keyboard input.
- Functions: Reusable blocks of code that perform specific tasks.
- Browser APIs: Using built-in browser features like localStorage for persistent data storage.
- Validation: Checking user input before processing (e.g., email format, field length).

Code Snippet Theme Toggle

```
/* THEME TOGGLE */
function initTheme() {
  const btn = document.getElementById("themeBtn");
  if (!btn) return;
  const icon = btn.querySelector("i");

  const apply = (theme) => {
    document.documentElement.setAttribute("data-theme", theme);
    localStorage.setItem("dp-theme", theme);
    if (icon) {
      icon.className = theme === "dark"
        ? "bi bi-sun-fill" : "bi bi-moon-stars-fill";
    }
  };

  apply(localStorage.getItem("dp-theme") || "dark");
  btn.addEventListener("click", () => {
    const current = document.documentElement.getAttribute("data-theme");
    apply(current === "dark" ? "light" : "dark");
  });
}
```

Code Snippet — Form Validation & Counter

```
/* FIELD VALIDATION */
function validateField(inputId, msgId, testFn, errMsg) {
  const input = document.getElementById(inputId);
  const msg = document.getElementById(msgId);
  if (!input || !msg) return false;
  const val = input.value;
  const ok = testFn(val);
  input.className = "field-input " + (ok ? "is-valid" : "is-invalid");
  return ok;
}
```

```

}

/* COUNTER */
let counterVal = 0;
function adjustCounter(delta) {
  counterVal = Math.max(-99, Math.min(99, counterVal + delta));
  const el = document.getElementById("counterDisplay");
  if (el) {
    el.textContent = counterVal;
    el.style.color = counterVal > 0 ? "var(--emerald)"
      : counterVal < 0 ? "var(--rose)" : "var(--accent)";
  }
}

```

Explanation of Code

Theme Toggle:

- `document.getElementById('themeBtn')`: Finds the button element with ID 'themeBtn'.
- `setAttribute('data-theme', theme)`: Sets a root HTML attribute that CSS uses to switch themes.
- `localStorage.setItem('dp-theme', theme)`: Persists the user's theme choice across page reloads.
- `btn.addEventListener('click', ...)`: Attaches a click event handler to the button.

Form Validation:

- `testFn(val)`: Calls a validation test function returning true/false.
- `input.className`: Changes the CSS class to display valid/invalid styling.

Counter:

- `Math.max(-99, Math.min(99, ...))`: Clamps the counter value between -99 and 99.
- `el.textContent`: Updates the displayed number directly in the DOM.
- Ternary operator: Assigns color conditionally based on counter value.

Demonstration of Understanding

- DOM Manipulation: Finding elements and changing their properties (`className`, `textContent`, `style`).
- Event Handling: Using `addEventListener` to respond to user clicks.
- Data Persistence: `localStorage` remembers user preferences between sessions.
- Conditional Logic: `if/else` statements and ternary operators for branching.
- Arrow Functions: Modern JavaScript syntax for concise function definitions.
- Validation: Real-world form validation with clear visual feedback.

Conclusion

JavaScript transforms static HTML and CSS into dynamic, interactive web applications. It enables real-time validation, user preference storage, DOM manipulation, and complex logic. Mastering JavaScript is essential for full stack development: it is used not only in browsers (frontend) but also in server-side development with Node.js (backend).